ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #5

SplatCraft v0.1.0

Pure-Rust toolkit for Gaussian splats, built on the ArtCraft 3-tier pattern.



v0.1.0	9 Crates	8530	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\splatcraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

Pure-Rust toolkit for Gaussian splats, built on the ArtCraft 3-tier pattern.

Install

Easiest: tell Claude. In Claude Code (or any coding agent with a shell) say:

> Install splatcraft from github.com/sandraschi/splatcraft

The agent follows the "Agent install steps" in INSTALL.md: it checks for a

release, otherwise builds from source with Rust, registers the MCP server, and verifies it.

It asks before installing anything large. Until the first GitHub release exists, this means a

source build (needs the Rust toolchain).

Other ways, all detailed in INSTALL.md: prebuilt zip, MCPB bundle for Claude

Desktop, build from source, manual MCP config. mcpb is not Python-only; its manifest supports

`binary` servers and splatcraft's bundle wraps the Rust `splatcraft-cli.exe` (validated and

packed, not yet install-tested in Claude Desktop). Training additionally needs ffmpeg, COLMAP

and Brush, which are not bundled.

Status: Day 4 code complete, first real training run pending. What works today, and

nothing else is claimed:

- Read and write binary PLY (3DGS layout, SH degree 0 to 3).
- Read SPZ v2 and v3, write SPZ v3. SPZ v4 (zstd) is rejected with a clear error.
- `inspect`: count, bounding box, SH degree, opacity and scale stats, non-finite count.
- `convert`: PLY <-> SPZ, atomic write, no overwrite unless asked.
- `engine_status`: honest up/down for brush, ns-train, colmap, ffmpeg, splat-transform, bunx.

Looks at env overrides, PATH and the tools dir; nothing is executed or version-checked.

- `edit`: ordered step pipeline: `crop` (box, optional invert), `prune` (min opacity, max scale,

non-finite), `transform` (translate, uniform scale, axis-angle rotation), `reduce_sh`.

Edits run in RDF. Rotating splats that carry SH degree > 0 is refused, because SH rotation

is not built; run `reduce_sh` to degree 0 first.

- `merge`: 2 to 64 files, mixed PLY/SPZ, frames unified, SH reduced to the lowest degree.
- `render`: headless GPU PNG still (wgpu-3dgs-viewer, auto-framed orbit camera, opaque black

background). Reports `unsupported` when no GPU adapter exists.

- `worlds`: World Labs Marble client (absorbs worldlabs-mcp): `ledger`, `credits`, `generate`, `status` (optional wait), `fetch` (SPZ splats, collider mesh, pano, thumbnail), `list`.

Request shapes follow the working Python bridge. Tested against a scripted fake transport only: no live call has been made yet, and the file upload path (prepare_upload then PUT) is

untested against the real service.

- `train`: video, image folder or COLMAP dataset to splat (absorbs splatmaker-mcp). Actions

`plan` (dry run, shows every stage), `start`, `status`, `log`, `cancel`, `list`. Details below.

- Live viewport in the egui app: type a path, press Load, drag to orbit, wheel to zoom, double-click to reset. It draws through an egui-wgpu paint callback using the same `LiveScene` code as the headless `render`. Verified so far: it compiles, the app starts on the wgpu backend, and the shared scene path passes the GPU render test. Mouse feel (drag

direction, zoom speed) and colour on the window surface have not been judged by eye yet.

- MCP stdio server and CLI over one dispatch bus; egui host with `--smoke`.

Spend guard for `worlds`: `generate` is a dry run unless `confirm_spend` is true. Each confirmed submit (about 1500 API credits for `marble-1.1`) is appended to `marble-ledger.jsonl` in `SPLATCRAFT_DATA` (default `./data`, gitignored) only after the POST succeeds. A cumulative cap (`SPLATCRAFT_MARBLE_CAP`, default 4500) refuses further submits; a per-call `max_credits` can lower it but never raise it. The key comes from `WORLDLABS_API_KEY` or, if that is unset, from the Windows Credential Manager (store it with `splatcraft-cli secret set worlds`, which prompts with hidden input). `secret status` shows

whether a key is set and where from, never the value. There is no MCP tool or bus op that can

read or write secrets, and the key is never returned or logged. See docs/SECRETS_STANDARD.md. API credits are separate from the

marble.worldlabs.ai web credits (check platform.worldlabs.ai).

Training (`train`)

The Rust code plans, launches and supervises; the heavy work is done by external programs:

ffmpeg (frame extraction), COLMAP (structure from motion) and Brush (the trainer, Rust).

Nerfstudio (Python) and splat-transform (via bunx) are only detected, never required.

- Inputs: exactly one of `video`, `images_dir`, `dataset`. Plus `output` (.ply or .spz),

`name`, `overwrite`, `target_frames` (150), `max_resolution` (1600), `iters` (10000),

`max_splats` (2000000), `sh_degree` (3), `quality` ("medium").

- Flags are discovered from each tool's `--help` and only seen flags are emitted, so Brush

builds with `--total-steps` (v0.3.x) and newer ones with `--total-train-iters` both plan.

- `start` spawns a detached supervisor (`splatcraft-cli job-run`) that runs the stages with no

shell, per-stage timeouts, a heartbeat every 5 s and a cancel marker. Jobs survive the caller exiting. A job whose heartbeat is stale for 45 s reports `lost`; one still queued after 60 s reports `not_started`.

- Layout: `<data>/jobs/<id>/{job.json,log.txt,cancel}`, event log `<data>/jobs.jsonl`, work dirs `<data>/work/<name>-<unix>`.

- Status: planning, process control, timeout kill and cancel are tested with real processes.

No real video-to-splat run has completed yet. Plan against real Brush v0.3.0 and COLMAP 4.2.1 resolves correctly. Orbit footage of a real object is needed; generated or animated clips will most likely fail at the COLMAP sparse-model check. Brush wants VRAM headroom.

Where the tools are found (first hit wins): env override `SPLATCRAFT_BRUSH`, `_NS_TRAIN`, `_COLMAP`, `_FFMPEG`, `_FFPROBE`, `_SPLAT_TRANSFORM`, `_BUNX` (a set but wrong override means "not found", no fallback), then PATH, then the tools dir `SPLATCRAFT_TOOLS` (default `./tools`; `<tool>/bin` then `<tool>`). `SPLATCRAFT_CLI` overrides the supervisor binary.

Not built yet: collision mesh generation of our own, texturecraft recolor of monochrome splats, SH rotation. See the plan below.

Use

```
just ci                                  # fmt, clippy, tests
cargo run -p splatcraft-mcp -- inspect --path scene.ply
cargo run -p splatcraft-mcp -- convert --input scene.ply --output scene.spz
cargo run -p splatcraft-mcp -- engines
cargo run -p splatcraft-mcp -- edit --input a.ply --output b.spz --steps "@steps.json"
cargo run -p splatcraft-mcp -- merge --inputs a.ply b.spz --output m.ply
cargo run -p splatcraft-mcp -- render --input a.ply --output a.png --yaw-deg 30
cargo run -p splatcraft-mcp -- worlds --args "@dry.json"   #
{"action":"generate","prompt":"…","…":"…"} is a dry run
cargo run -p splatcraft-mcp -- train --args "@plan.json"   #
{"action":"plan","video":"clip.mp4","output":"out.ply"}
cargo run -p splatcraft-mcp -- mcp        # MCP stdio server
cargo run -p splatcraft --bin splatcraft  # desktop host
```

`--steps` and `--args` take inline JSON or `@file.json` (use the file form in PowerShell, which strips inner double quotes).

MCP tools: `splat_inspect`, `splat_convert`, `splat_edit`, `splat_merge`, `splat_render`, `splat_worlds`, `splat_train`, `splat_engine_status`, `splat_synth`.

Set `SPLATCRAFT_ROOT` to confine all paths to a directory.

Plan (AI-assisted days)

Day	Scope
1	Workspace, PLY and SPZ codecs, inspect, convert, engine probe, gates (done)
2	Edit ops (crop, transform, merge, prune), headless render (done); live egui viewport (done, needs a look by eye)
3	Marble client (absorbs worldlabs-mcp), spend ledger (done, offline-tested; first live read-only check pending)
4	Brush trainer adapter, ffmpeg and COLMAP prep, job ledger (done, tested with real processes); first real video-to-splat run pending a proper orbit clip
5	Collision mesh, texturecraft bridge spike
6 to 7	Recolor of grey splats, democraft recipes, installer

License

MIT OR Apache-2.0.

Product Requirements & Architecture Specification

Repo: `splatcraft` (Pure-Rust ArtCraft pattern, 9 crates)

Version: 0.1.0 (workspace.package)

Transport: MCP 2024-11-05 stdio (`splatcraft-cli mcp`); no control port claimed

Crates: `splat-formats` (PLY/SPZ codecs) / `splat-ops` (crop, prune, transform, merge, reduce_sh) / `splat-view` (`LiveScene` via wgpu-3dgs-viewer 0.8) / `splat-worlds` (Marble client, fake-transport tests) / `splat-train` (tool locator, planner, supervisor) / `craft-secrets` (env first, OS store second) / `splatcraft-core` (L1 `dispatch`) / `splatcraft-mcp` (L2 CLI+MCP, hidden `job-run`) / `splatcraft-app` (L3 egui viewport)

1. Problem

Gaussian-splat work is split across converters, viewers, the Marble web credit flow, and trainer CLIs (ffmpeg, COLMAP, Brush) with incompatible flags. A single local bus should inspect, convert, edit, merge, render, generate, and train splats without moving secrets through the LLM context.

2. What works today (nothing else is claimed)

- Binary PLY read/write (3DGS layout, SH degree 0-3); SPZ v2/v3 read, v3 write; SPZ v4 (zstd) refused with a clear error.
- `inspect`, `convert` (atomic write, no overwrite unless asked), `engine_status` (env, PATH, tools dir; nothing executed).
- `edit`: crop, prune, transform (translate, uniform scale, axis-angle rotation), `reduce_sh`, all in RDF. Rotating SH degree > 0 is refused; run `reduce_sh` to degree 0 first.
- `merge`: 2-64 files, mixed PLY/SPZ, SH reduced to the lowest degree.
- `render`: headless GPU PNG still (opaque black); `unsupported` with no GPU adapter.
- `worlds`: Marble ledger/credits/generate/status/fetch/list behind a `Transport` trait; scripted fake only, no live call yet. Spend guard: dry run unless `confirm_spend` is true, JSONL ledger after POST, cap `SPLATCRAFT_MARBLE_CAP` (default 4500), per-call `max_credits` can only lower it. See `SECRETS_STANDARD.md`: no MCP tool or bus op touches secrets; only `splatcraft-cli secret set|status|delete`.
- `train`: plan/start/status/log/cancel/list over ffmpeg + COLMAP + Brush with `--help` flag discovery, detached supervisor, heartbeat 5 s, stale 45 s means `lost`. Layout `<data>/jobs/<id>/`, event log `jobs.jsonl`.
- L3 viewport: path box, Load, orbit drag, wheel zoom, double-click reset through the same `LiveScene` as headless `render`.

3. Non-goals

No SH rotation math before `reduce_sh`; no live Marble spend without owner confirmation; no committed binaries or engines in the repo (`SPLATCRAFT_TOOLS` lives outside, Goliath default `D:\Dev\tools\splatcraft`); no second trainer before Brush flags are stable.

4. Rules (from AGENTS.md, binding here too)

LF line endings only; no em dashes in docs, code, or commits; realistic day-scale timelines; single trunk `master` (short-lived `agent/<task>` branches only, merged and deleted same day); install requests follow `INSTALL.md` agent steps exactly.

5. Gates

`just fmt-check`, `just check`, `just test`, `just build-release`, then `splatcraft --smoke`. `just ci` runs the first three. Render tests skip with a stderr note when no GPU adapter exists.